

ОСНОВЫ

Application Security

Безопасность приложений

№2

Степан Трифонов

Специалист по безопасности приложений
Защищая рекламные продукты VK



Цель занятия:

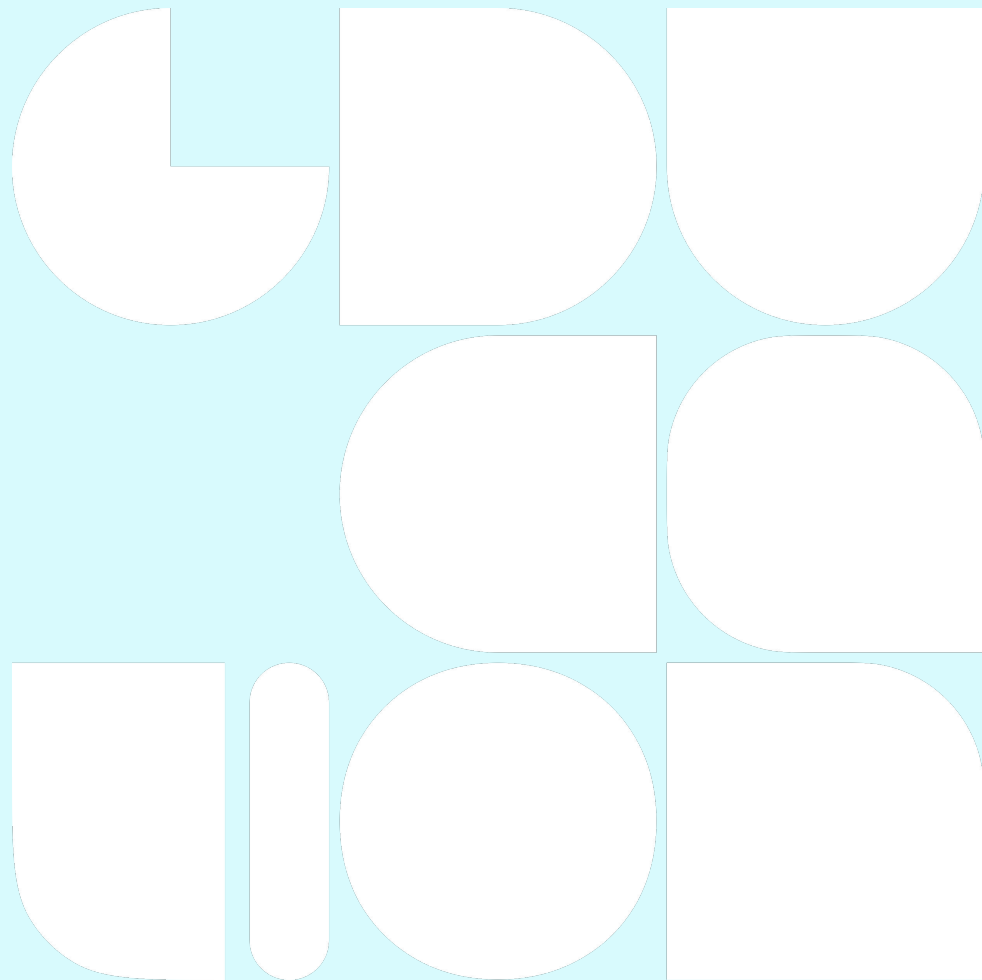
Сформировать базовое понимание принципов Application Security (AppSec), роли AppSec-инженера и процесса SSDLC (Secure Software Development Lifecycle)



О чём поговорим?

- Какие существуют концепции и практики в области безопасности приложений
- Что такое Application Security и зачем оно нужно
- Какая инструментальная составляющая безопасности приложений
- Как развиваться и что для этого делать в безопасности приложений (и в информационной безопасности в целом)

Стандарты и документация



Стандарты и документация

→ ISO/IEC 27000 series:

Серия стандартов Международной организации по стандартизации (ISO) в области информационной безопасности, включая ISO 27001 (система менеджмента информационной безопасности), ISO 27002 (рекомендации по контрольным мерам), и другие

→ NIST Cybersecurity Framework:

Фреймворк, разработанный NIST, предоставляет рекомендации по управлению кибер-рисками

Стандарты и документация

→ CIS Controls:

Контрольные списки, разработанные CIS, предоставляют лучшие практики для защиты IT-систем

→ COBIT:

Фреймворк для управления и аудита ИТ, охватывающий различные аспекты, включая безопасность

→ ITIL:

Фреймворк для управления ИТ-услугами, включающий аспекты безопасности

→ PCI DSS (Payment Card Industry Data Security Standard):

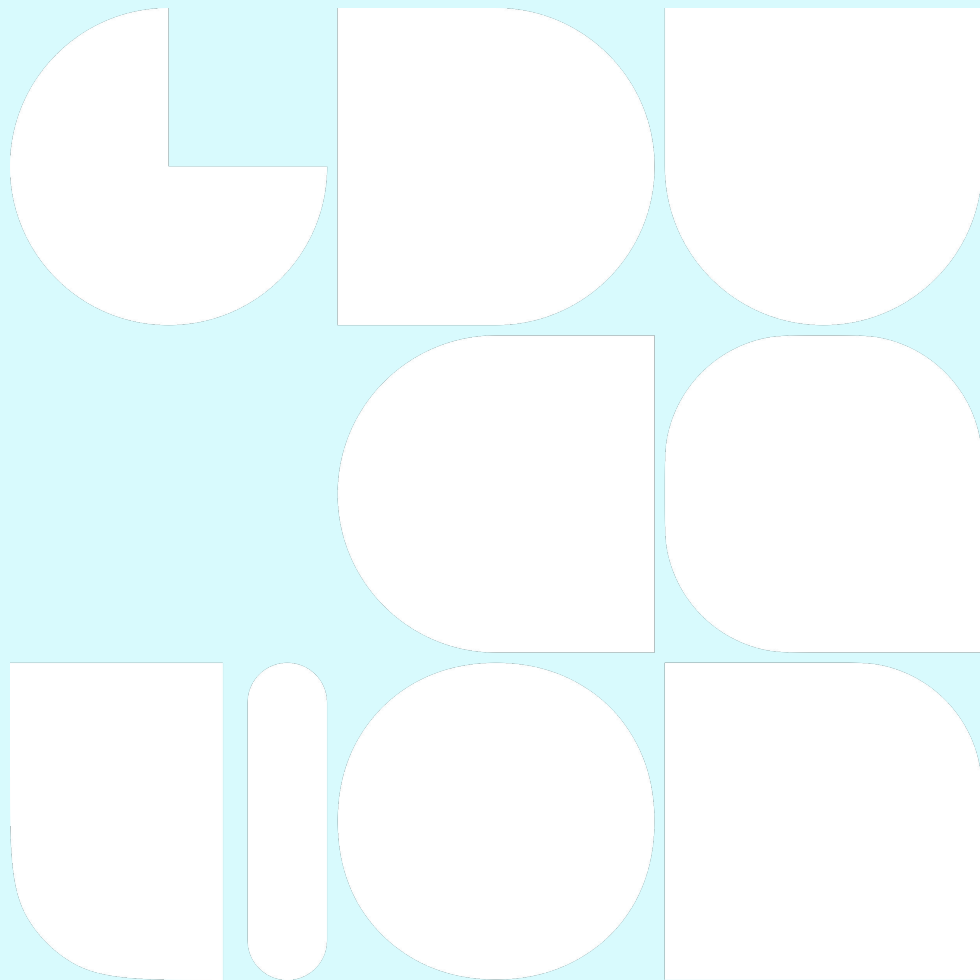
Стандарт безопасности данных индустрии платежных карт

Стандарты и документация

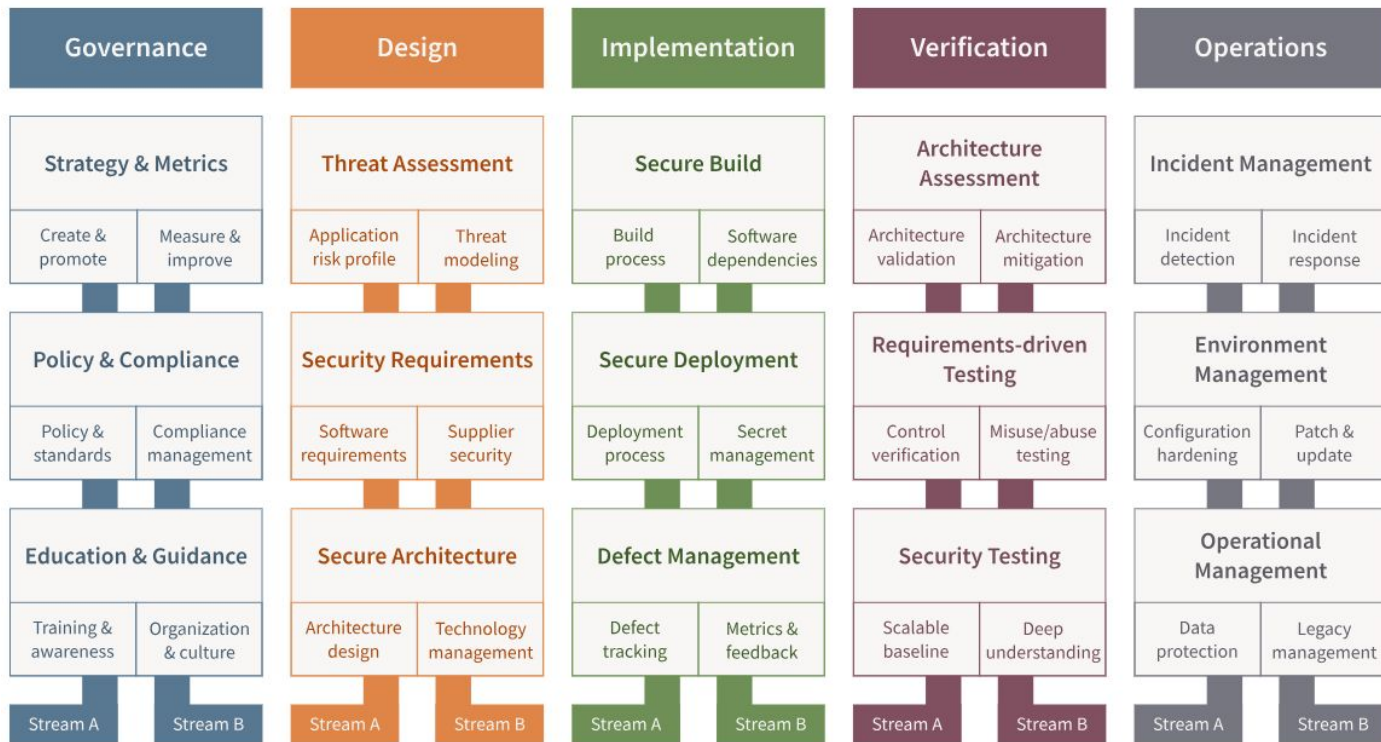
- 149-ФЗ «Об информации, информационных технологиях и о защите информации»
- 152-ФЗ «О персональных данных»
- Требования по защите коммерческой тайны (98-ФЗ)
- Требования по защите инсайдерской информации (224-ФЗ)
- Требования по защите данных в Национальной платежной системе (161-ФЗ, ПП-584, 382-П и другие)
- Приказ ФСТЭК России №17 («Об утверждении Требований о защите информации, не составляющей государственную тайну, содержащейся в государственных информационных системах»), №21 («Об утверждении Составов и содержания организационных и технических мер по обеспечению безопасности персональных данных при их обработке в информационных системах персональных данных»)
- Постановление Правительства РФ №1119
- Требования по защите информации в АСУ ТП (Приказ ФСТЭК России №31)

ОСНОВЫ

- Что есть AppSec? А DevSecOps?
- Зачем это вообще нужно?
- Какое место в жизненном цикле разработки занимает?



Процессы



Что такое Application Security?

Application Security (AppSec) — это практика защиты приложений от уязвимостей и атак

Включает в себя методологии, инструменты и процессы, направленные на выявление, предотвращение и устранение угроз безопасности на всех этапах разработки

Основные задачи:

- Анализ кода на уязвимости
- Безопасное проектирование
- Тестирование безопасности
- Мониторинг и реагирование

Что такое DevSecOps?

DevSecOps — это интеграция принципов безопасности (Security) в процессы разработки (Development) и эксплуатации (Operations)

В отличие от традиционного подхода, где безопасность проверяется в конце разработки, DevSecOps встраивает ее на каждом этапе SDLC

Основные принципы:

- Автоматизация проверки безопасности
- Совместная ответственность между командами Dev, Sec и Ops
- Быстрое выявление и устранение уязвимостей

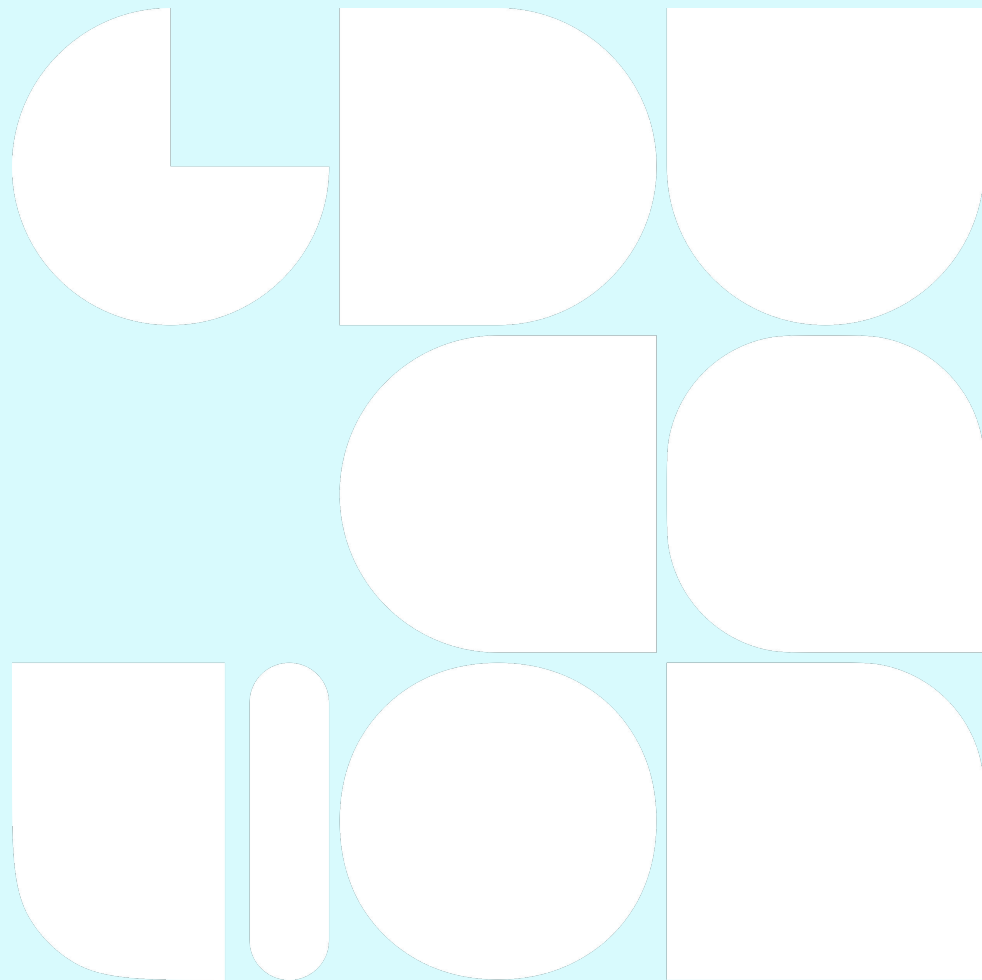
Зачем вообще нужна Application Security?

- **Минимизация рисков** — предотвращение утечек данных, атак и репутационных потерь
- Соответствие **нормативным требованиям** (GDPR, ISO 27001, PCI DSS и др.)
- **Защита пользователей** — обеспечение безопасности их данных и предотвращение эксплуатации уязвимостей
- **Экономия ресурсов** — исправление уязвимостей на ранних этапах дешевле, чем после выхода продукта
- **Доверие клиентов** — безопасные приложения повышают лояльность и конкурентоспособность

Методология

- BSIMM (Building Security In Maturity Model)
- OWASP SAMM (Software Assurance Maturity Model)
- Microsoft SDL (Security Development Lifecycle)
- NIST SSDF (Secure Software Development Framework)
- SANS Software Security Framework

Практики и инструменты: SAST



Практики и инструменты: SAST

Что это такое?

Тестирование «белым ящиком»

- Анализ статический — запуск приложения не требуется
- Различные виды анализа:
 - ◆ Pattern-based
 - ◆ Data flow
 - ◆ Taint
 - ◆ Symbolic
- Позволяет обнаруживать уязвимости на ранних этапах разработки

Практики и инструменты: SAST

Как это работает?

- Сбор «промежуточного представления»
- Лексический анализ и преобразование во внутреннее представление
- Синтаксический анализ — для определения структуры кода и его связей
- Анализ потока управления — для определения какие части кода могут быть достигнуты при выполнении программы
- Анализ данных — для определения возможных уязвимостей, связанных с обработкой данных
- Поиск уязвимостей – по соответствующим паттернам или аномалиям
- Генерация отчета

Практики и инструменты: SAST

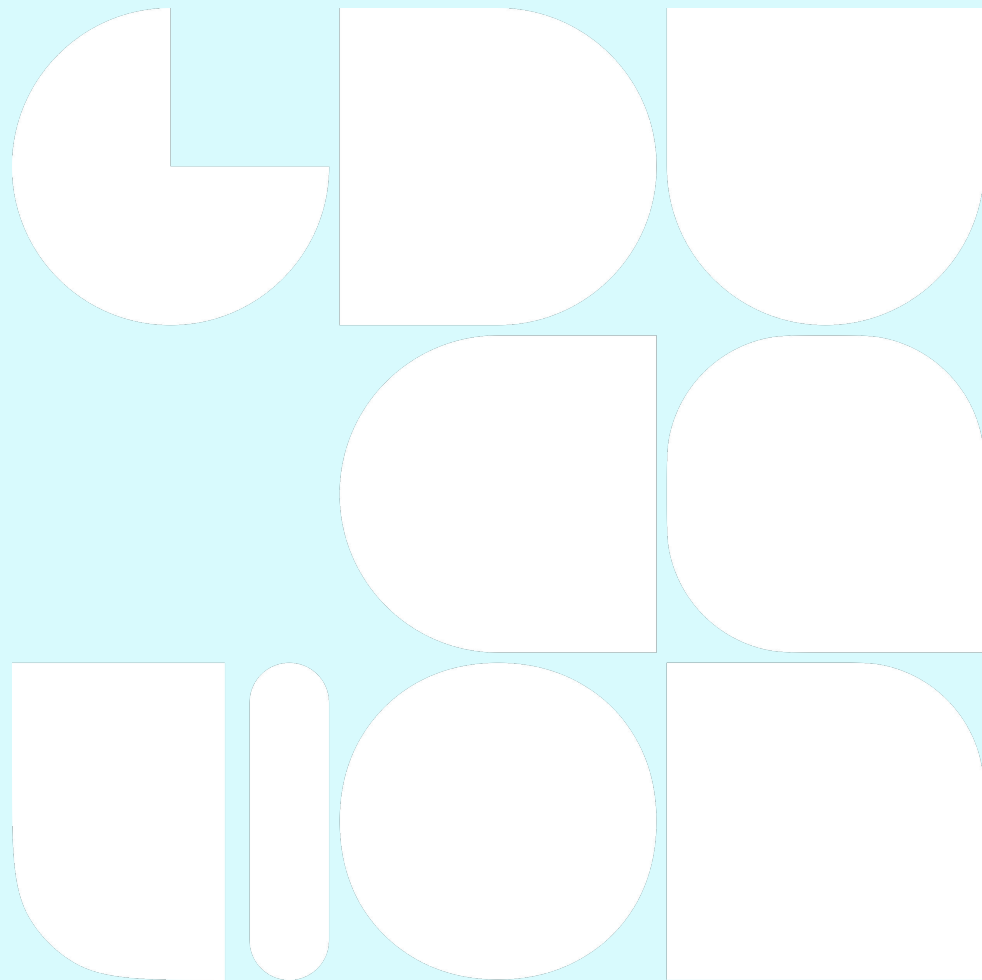
Плюсы:

- Раннее обнаружение уязвимостей
- Большое покрытие исходного кода
- Простота интеграции в CI/CD

Минусы:

- Большое количество ложных срабатываний
- Ограничения на язык программирования/фреймворк
- Невозможность обнаружения некоторых классов уязвимостей

Практики и инструменты: OSA/SCA



Практики и инструменты: OSA/SCA

Что это такое?

OSA:

- Анализ OpenSource компонентов
- Плагины для артефакториев/ прокси-репозиторийев



До начала фактического использования

SCA:

- Анализ состава ПО
- Утилиты сканирования + базы данных уязвимостей



Во время использования в проектах

Практики и инструменты: OSA/SCA

Что это дает?

Возможность контроля:

- Публично известных уязвимостей
- Лицензионной чистоты использования компонентов
- Состава приложения или продукта

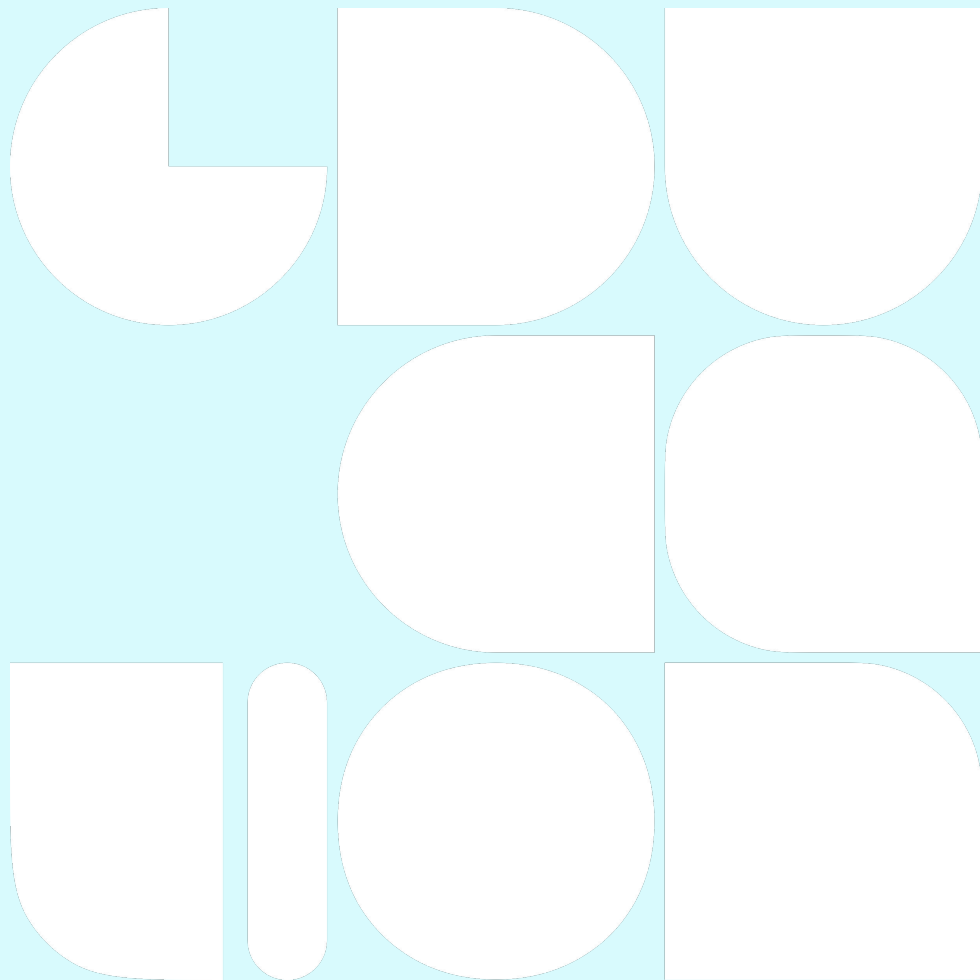


Практики и инструменты: OSA/SCA

Как это работает?

- Сбор списка и построение структуры зависимостей и информации о лицензиях:
 - Генерация SBOM-файла
 - Парсинг lock-файлов
 - Сбор информации из сборки продукта (декомпиляция)
- Анализ зависимостей по информации в базах данных уязвимостей (feeds)
- Генерация отчета

Практики и инструменты: Secret Scanning



Практики и инструменты: Secret Scanning

Что это такое?

- Анализ исходного кода на наличие чувствительной информации (пароли, токены)
- Различные методы анализа:
 - ◆ Энтропия
 - ◆ Regex
 - ◆ Семантический
 - ◆ Комбинированные техники
- Возможность поиска в истории репозитория

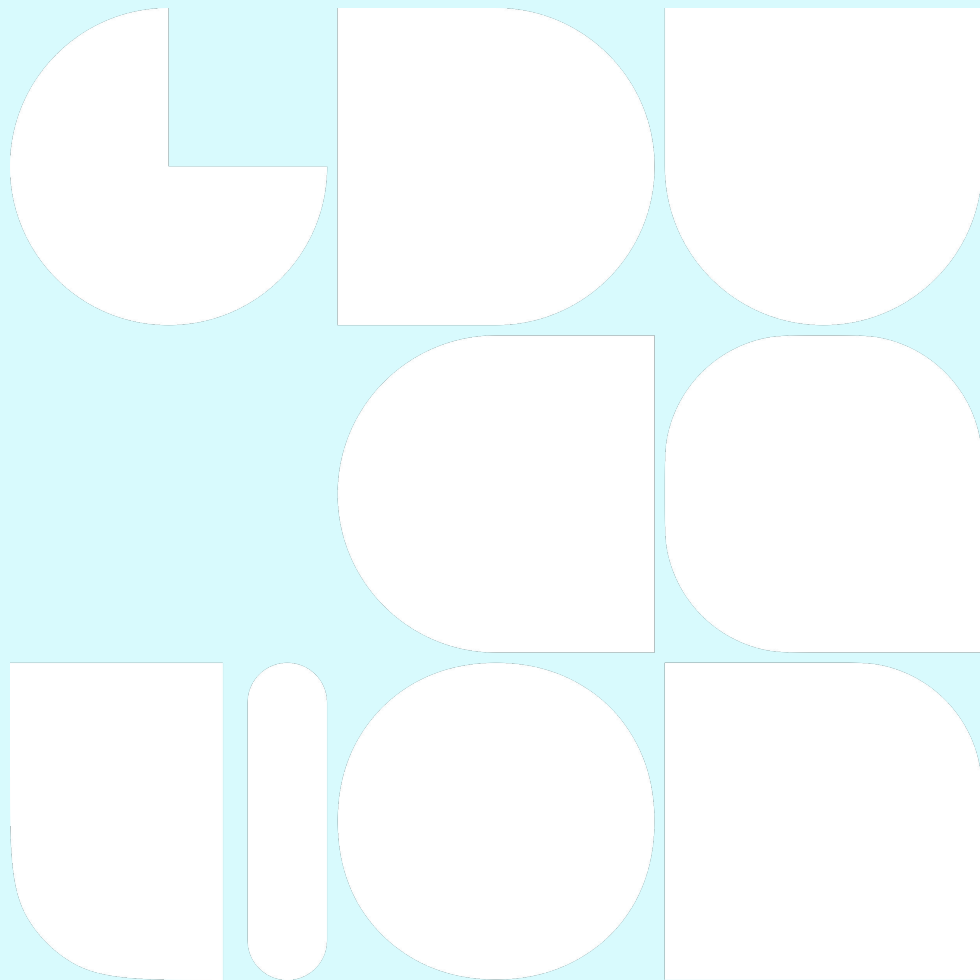
Практики и инструменты: Secret Scanning

Как это работает?

- Запуск инструмента на исходном коде или Git-репозитории
- Генерация отчета



Практики и инструменты: DAST



Практики и инструменты: DAST

Что это такое?

Тестирование «черным ящиком» («серым ящиком»)

- Анализ динамический — требуется запуск приложения
- Различные подходы:
 - ◆ Имитация реальных атак
 - ◆ Тестирование параметров
 - ◆ Fuzzing
 - ◆ Анализ ошибок
 - ◆ Манипуляция сессиями
 - ◆ Отказ в обслуживании
- Функциональные тесты приложения могут быть конвертированы в DAST-проверки
- Некоторые виды уязвимостей могут быть найдены только при динамическом анализе

Практики и инструменты: DAST

Как это работает?

- Сбор информации о приложении
 - ◆ Краулинг (URL, страницы, API-эндпоинты и т.д.)
 - ◆ Анализ API-спецификаций (исходного кода)
 - ◆ Построение карты приложения
- Проверки исследуемого приложения
- Анализ ответов
- Генерация отчета

Практики и инструменты: DAST

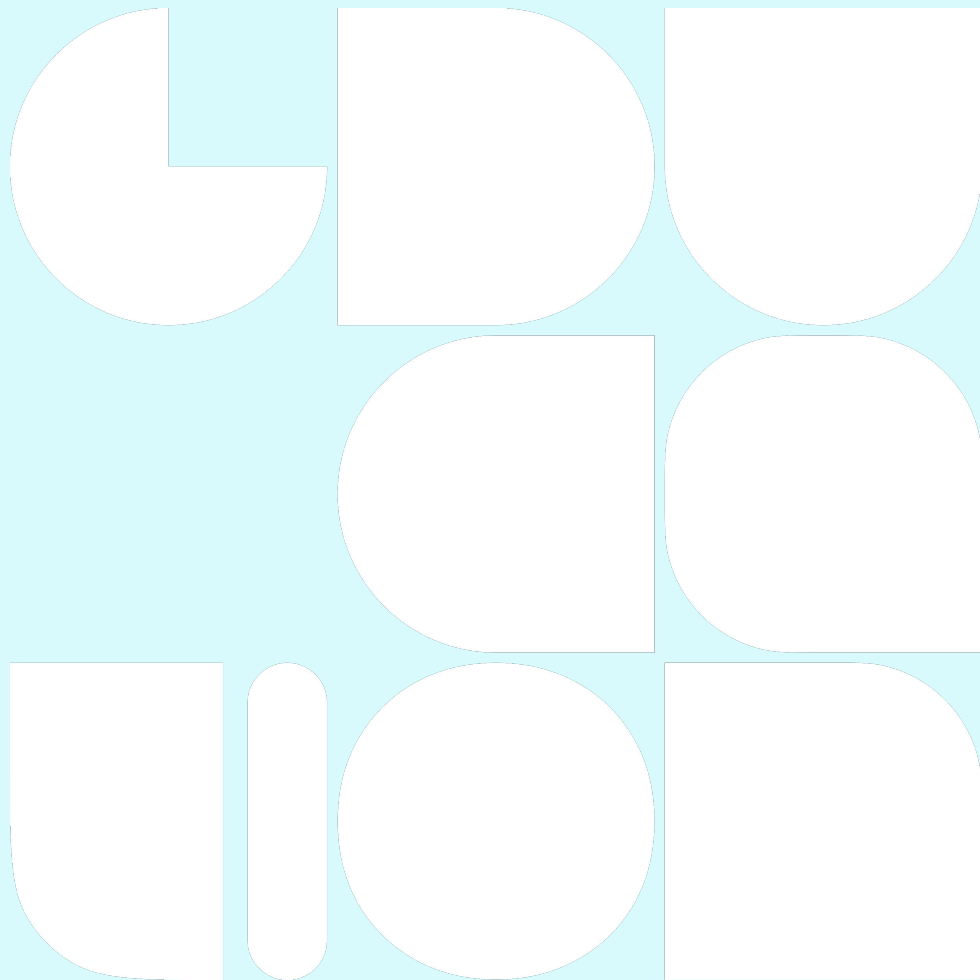
Плюсы:

- Реалистическая оценка уязвимостей (готовый PoC)
- Сравнительно низкое количество ложных срабатываний
- Отсутствие маппинга уязвимостей на исходный код

Минусы:

- Трудоемкость внедрения
- Позднее обнаружение уязвимостей
- Временные затраты на проведение проверок
- Отсутствие маппинга уязвимостей на исходный код

Практики и инструменты: Container Security



Практики и инструменты: Container Security

Что это такое?

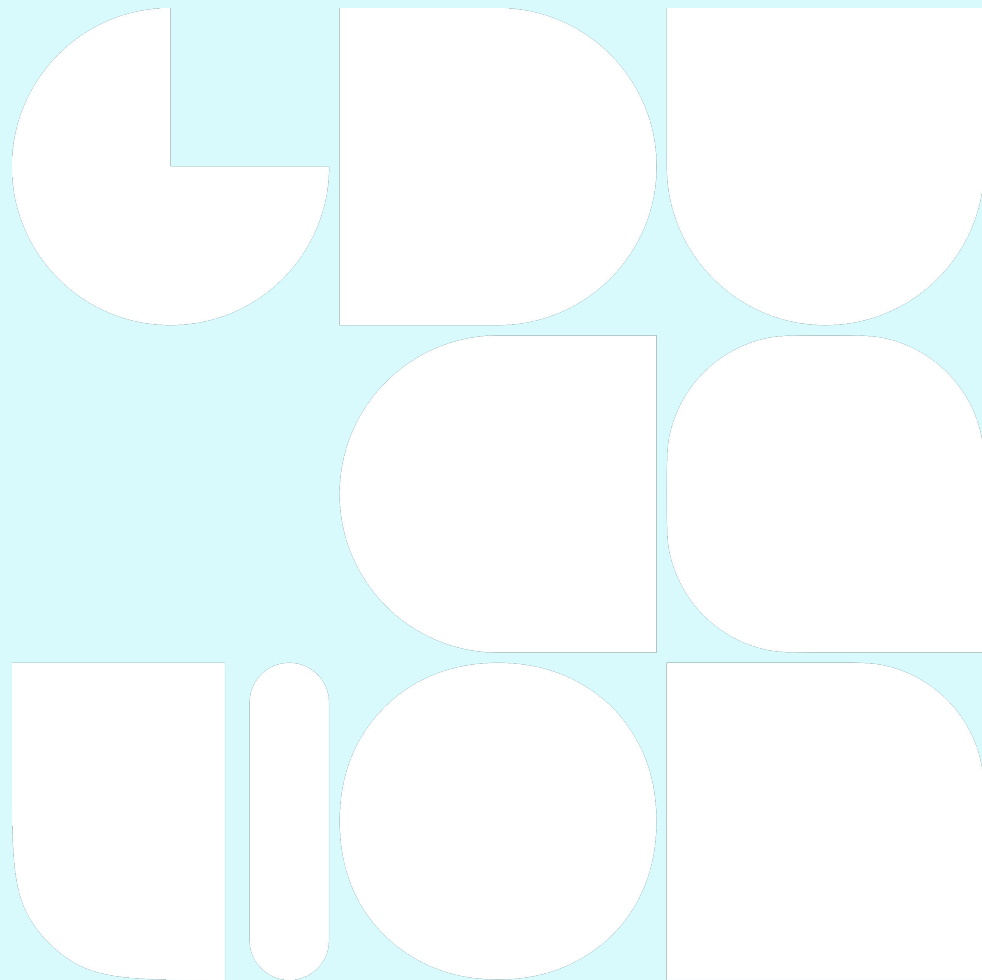
- Комплексный подход к обеспечению безопасности технологий контейнеризации:
 - ◆ Манифесты и файлы конфигураций: Everything as Code
 - ◆ Opensource-компоненты в составе образов (SCA)
 - ◆ Соответствие Compliance-требованиям: CIS*, PCI, HIPAA
 - ◆ Контроль целостности артефактов (подпись, аудит и т.д.)
 - ◆ Безопасность настроек оркестраторов (Docker, k8s и т.д.)
 - ◆ Безопасность среды выполнения (Runtime)
- Различные инструменты для точечных проверок или платформа комплексных решений

Практики и инструменты: Container Security

Как это работает?

- Анализ состава образов (пакеты ОС, программные компоненты)
- Анализ манифестов (паттерны безопасности, best practices)
- Анализ конфигураций (развертывания, настроек оркестратора и ОС)
- Постоянный анализ среды развертывания и выполнения (сетевые политики, ролевая модель, политики безопасности)
- Генерация отчетов (и отображение статистики)

Практики и инструменты: Penetration Testing



Практики и инструменты: Penetration Testing

Что это такое?

- Анализ защищенности
«черным/серым/белым ящиком»
(в контексте AppSec)
- Неотъемлемая часть аудита безопасности
- Имитация реальной атаки
(в production-условиях)
- Проверка применимости CVE (PoC)
- Способ проверки эффективности AppSec-мер (и не только)
- Поиск и эксплуатация сложных уязвимостей



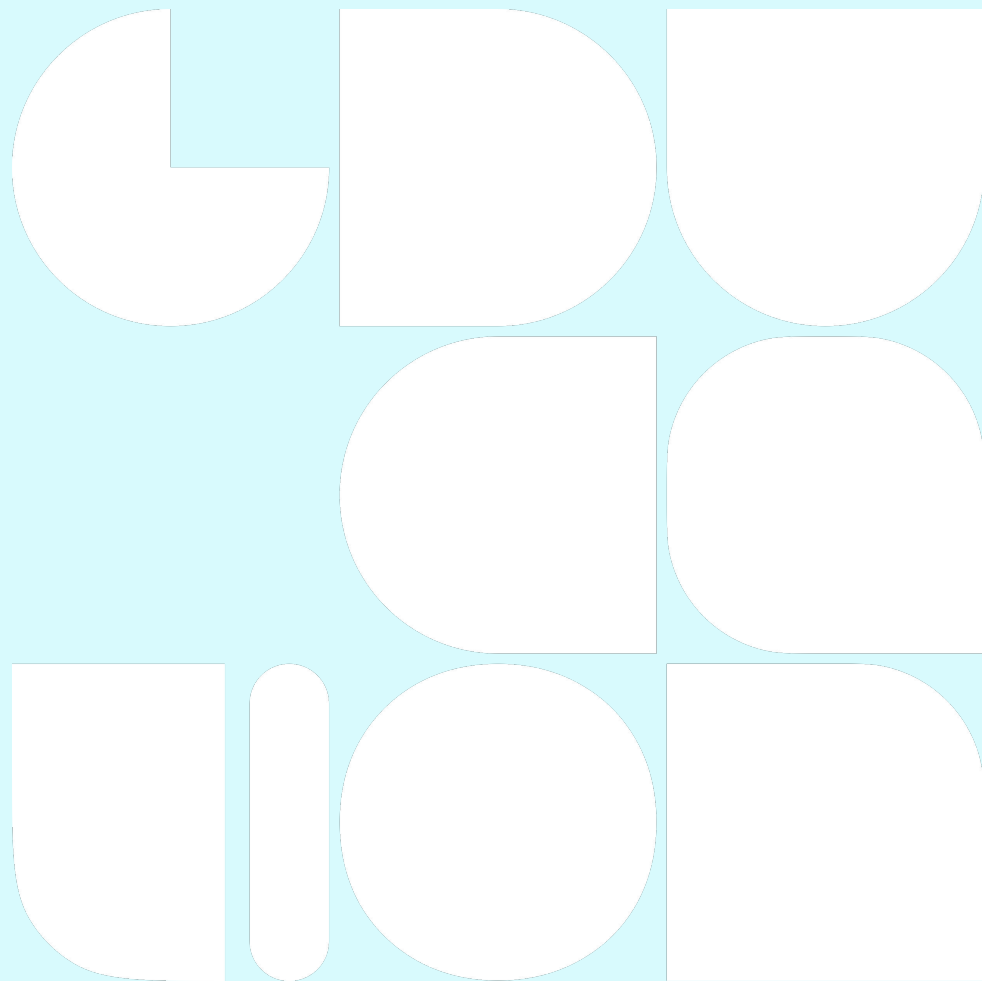
Практики и инструменты: Penetration Testing

Как это работает?

- Определение целей и объема тестирования
- Сбор информации о приложении (архитектура, технологический стек и т.д.)
- Анализ «слабых» мест и потенциальных векторов атак
- Эксплуатация уязвимостей
- Оценка рисков
- Подготовка отчета
- Реагирование (виртуальный патчинг, доработка инструментов, исправление уязвимости)



Как этим всем
управлять?



Как этим всем управлять?

Интегрируем инструменты:

- CI/CD пайплайны
- Использование Git Hooks
- ASOC-решения (aka ASPM)

Как этим всем управлять?

Взаимодействуем с командами разработки:

- Security Champions
- Security Quality Gates
Метрики SLA и состояния безопасности
- Обучение(Security Awareness)

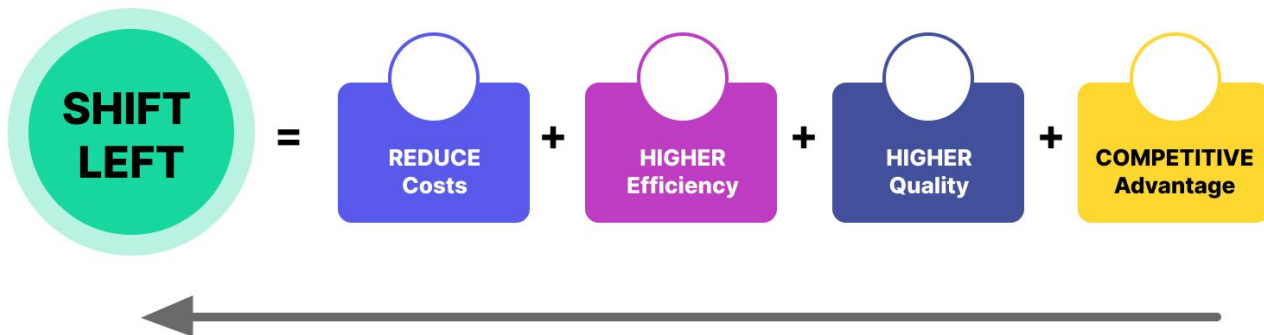


Как этим всем управлять?

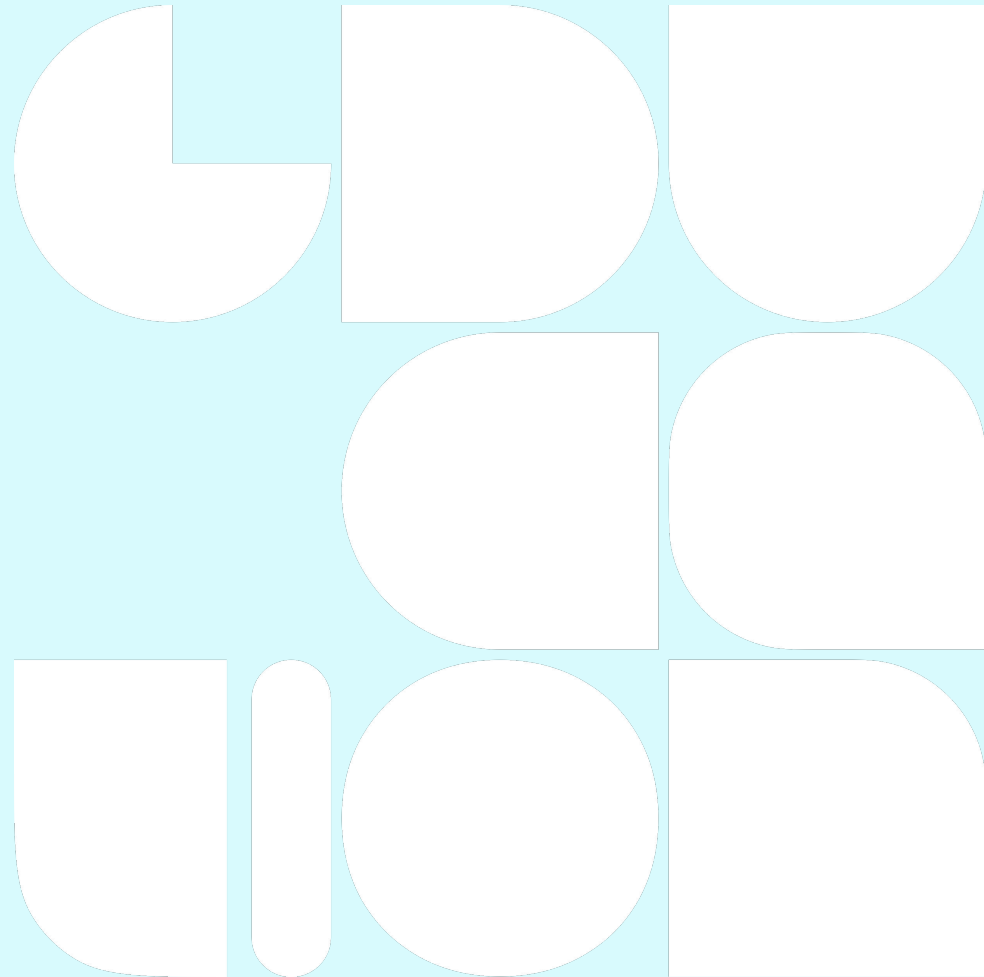
Оптимизируем процессы AppSec:

Shift-Left (Shift-Everywhere)

- Доработка и тюнинг инструментов
- Работа с рисками безопасности
- Периодическая оценка уровня зрелости



ASOC



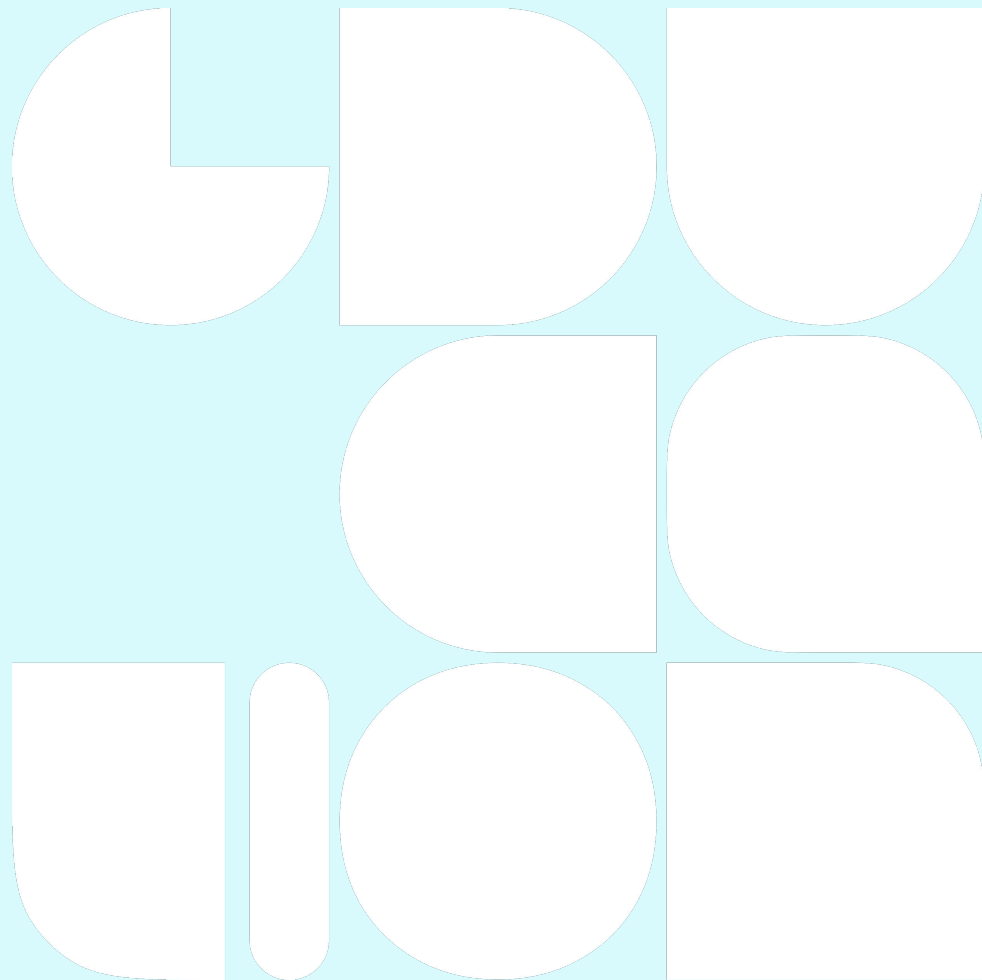
ASOC

Что это такое?

- Комплексный и централизованный подход к DevSecOps
- Платформа для решения задач:
 - ◆ Интеграции инструментов AppSec в жизненный цикл разработки
 - ◆ Оркестрации инструментов AppSec для запуска сканирований
 - ◆ Корреляции отчетов инструментов
 - ◆ Менеджмента уязвимостей
 - ◆ Настройки политик безопасности
 - ◆ Построения метрик



Как расти
в AppSec'e
и где учиться?



Как расти в AppSec'е и где учиться?

Уязвимые приложения:

→ DVWA

→ OWASP JuiceShop

→ k8s_goat

Как расти в AppSec'е и где учиться?

Тренировочные площадки и CTF:

- [HackTheBox](#)
- [TryHackMe](#)
- [Portswigger Academy](#)



HACKTHEBOX

Как расти в AppSec'е и где учиться?

Профильные мероприятия:

- PHDays
- Offzone
- Security-митапы и сообщество

Подведем итоги:

AppSec — защита приложений от уязвимостей и атак, направленная на минимизацию рисков, соблюдение нормативных требований, защиту пользователей, экономию ресурсов и повышение доверия клиентов

Безопасная разработка — это интеграция комплекса мер безопасности на всех этапах жизненного цикла ПО

Использование стандартов и документации помогает следовать лучшим практикам и соответствовать нормативным требованиям

Эффективные процессы безопасности должны быть встроены в рабочие процессы разработки для минимизации рисков

SAST позволяет выявлять уязвимости на этапе написания кода

OSA/SCA обнаруживает уязвимости в используемых сторонних компонентах

Подведем итоги:

Secret Scanning помогает находить и устранять утечки секретных данных в коде

DAST тестирует приложение в режиме выполнения для выявления уязвимостей

Container Security обеспечивает безопасность контейнеризированных приложений и их окружений

Penetration testing имитирует реальные атаки для оценки реальной защищенности приложения

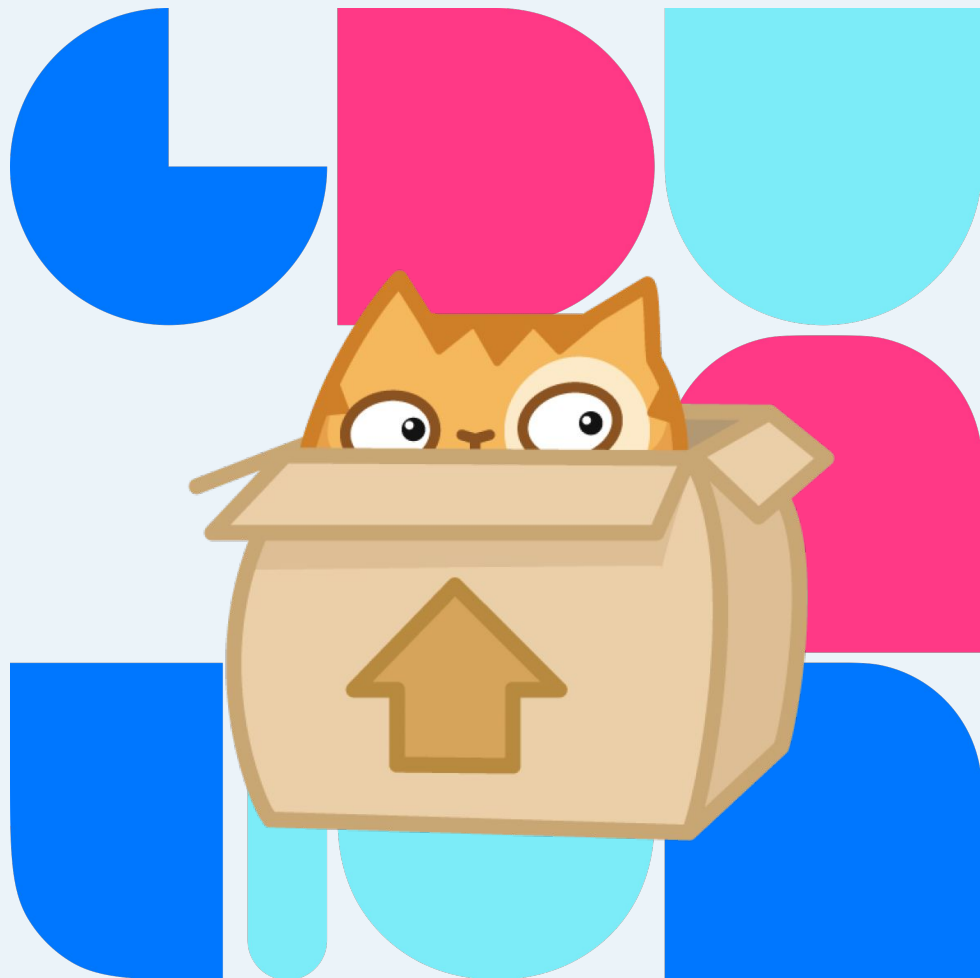
Управление безопасностью требует координации между командами и использования специализированных инструментов для мониторинга и реагирования на угрозы

Постоянное обучение, развитие практических навыков и разные сертификации в области безопасности приложений помогают оставаться актуальными в профессии и развивать карьеру

Дополнительный материал:

- OWASP Cheat Sheet Series — <https://cheatsheetseries.owasp.org>
- Stanford CS 253 Web Security — <https://web.stanford.edu/class/cs253/>
- Andrew Hoffman, Web Application Security: Exploitation and Countermeasures for Modern Web Applications
- Michal Zalewski, The Tangled Web: A Guide to Securing Modern Web Applications

Вопросы?



Спасибо за
внимание!

